Homework Assignment 8

Red-Black Trees

Assigned: Friday, November 6, 2026

Due: Friday, November 13, 2026 at 11:59pm

PROBLEM 1 12 POINTS

Implement insertion into a red-black tree in C++. You are provided with files `RedBlackTree.h` and `RedBlackTree.cpp` for a class RedBlackTree to use as a starting point. *You may add any additional data members and member functions, either private or public, to complete your implementation*. ***You do not need to implement any additional red-black tree functionality, such as search or node removal!***

You may use any pseudocode and code from Cormen book and the pseudocode and code provided in this assignment description for this problem. No other external sources are allowed!

Code snippet in `main()` function to test RBT tree rotations:

```
/*Test red-black tree inserts*/
RedBlackTree a;
RBTNode* aRoot = new RBTNode(22);
a.insert(aRoot);
a.insert(new RBTNode(11));
a.insert(new RBTNode(33));
a.insert(new RBTNode(55));
a.insert(new RBTNode(44));
std::cout << aRoot->key << std::endl;
std::cout << aRoot->left->key << " " << aRoot->right->key <<
std::endl;
std::cout << aRoot->right->left->key << " " << aRoot->right->right-
>key << std::endl;
```

Console output:

```
22
11 44
33 55
```

Submit your updated header file and implementation as `RedBlackTree.h`, `RedBlackTree.cpp`, **and** `avlRedBlackComparison.cpp` respectively!

The red-black balance operation consists of the steps below.

1. Assign `parent` with `node`'s parent, `uncle` with `node`'s uncle, which is a sibling of `parent`, and `grandparent` with `node`'s grandparent.
2. If `node` is the tree's root, then color `node` black and return.
3. If `parent` is black, then return without any alterations.
4. If `parent` and `uncle` are both red, then color `parent` and `uncle` black, color `grandparent` red, recursively balance `grandparent`, then return.
5. If `node` is `parent`'s right child and `parent` is `grandparent`'s left child, then rotate left at `parent`, assign `node` with `parent`, assign `parent` with `node`'s parent, and go to step 7.
6. If `node` is `parent`'s left child and `parent` is `grandparent`'s right child, then rotate right at `parent`, assign `node` with `parent`, assign `parent` with `node`'s parent, and go to step 7.
7. Color `parent` black and `grandparent` red.
8. If `node` is `parent`'s left child, then rotate right at `grandparent`, otherwise rotate left at `grandparent`.

Red-black tree insertion begins by calling `BSTInsert` to insert the node using the BST insertion rules. The newly inserted node is colored red and then a balance operation is performed on this node.

```
RBTreeInsert(tree, node) {
   BSTInsert(tree, node)
   node⇢color = red
   RBTreeBalance(tree, node)
}

RBTreeBalance(tree, node) {
   if (node⇢parent == null) {
      node⇢color = black
      return
   }
   if (node⇢parent⇢color == black)
      return
   parent = node⇢parent
   grandparent = RBTreeGetGrandparent(node)
   uncle = RBTreeGetUncle(node)
   if (uncle != null && uncle⇢color == red) {
      parent⇢color = uncle⇢color = black
      grandparent⇢color = red
      RBTreeBalance(tree, grandparent)
      return
   }
   if (node == parent⇢right &&
```

```
       parent == grandparent⇢left) {
      RBTreeRotateLeft(tree, parent)
      node = parent
      parent = node⇢parent
   }
   else if (node == parent⇢left &&
            parent == grandparent⇢right) {
      RBTreeRotateRight(tree, parent)
      node = parent
      parent = node⇢parent
   }
   parent⇢color = black
   grandparent⇢color = red
   if (node == parent⇢left)
      RBTreeRotateRight(tree, grandparent)
   else
      RBTreeRotateLeft(tree, grandparent)
}
```

The `RBTreeBalance` function uses the `RBTreeGetGrandparent` and `RBTreeGetUncle` utility functions to determine a node's grandparent and uncle, respectively.

```
RBTreeGetGrandparent(node) {
   if (node⇢parent == null)
      return null
   return node⇢parent⇢parent
}

RBTreeGetUncle(node) {
   grandparent = null
   if (node⇢parent != null)
      grandparent = node⇢parent⇢parent
   if (grandparent == null)
      return null
   if (grandparent⇢left == node⇢parent)
      return grandparent⇢right
   else
      return grandparent⇢left
}
```

The `RBTreeRotateLeft` function performs a left rotation at the specified node by updating the right child's left child to point to the node, and updating the node's right child to point to the right child's former left child. If non-null, the node's parent has the child pointer changed from node to the node's right child. Otherwise, if the node's parent is null, then the tree's root pointer is updated to point to the node's right child.

```
RBTreeRotateLeft(tree, node) {
   rightLeftChild = node⇢right⇢left
   if (node⇢parent != null)
      RBTreeReplaceChild(node⇢parent, node, node⇢right)
   else { // node is root
      tree⇢root = node⇢right
      tree⇢root⇢parent = null
   }
   RBTreeSetChild(node⇢right, "left", node)
   RBTreeSetChild(node, "right", rightLeftChild)
}
```

Right rotation is analogous to left rotation. A right rotation at a node requires the node's left child to be non-null.

```
RBTreeRotateRight(tree, node) {
   leftRightChild = node⇢left⇢right
   if (node⇢parent != null)
      RBTreeReplaceChild(node⇢parent, node, node⇢left)
   else { // node is root
      tree⇢root = node⇢left
      tree⇢root⇢parent = null
   }
   RBTreeSetChild(node⇢left, "right", node)
   RBTreeSetChild(node, "left", leftRightChild)
}
```

The `RBTreeSetChild` utility function sets a node's left child, if the whichChild parameter is "left", or right child, if the whichChild parameter is "right", and updates the child's parent pointer.

```
RBTreeSetChild(parent, whichChild, child) {
   if (whichChild != "left" && whichChild != "right")
      return false

   if (whichChild == "left")
      parent⇢left = child
   else
      parent⇢right = child
   if (child != null)
      child⇢parent = parent
   return true
```

The `RBTreeReplaceChild` utility function replaces a node's left or right child pointer with a new value.

```
RBTreeReplaceChild(parent, currentChild, newChild) {
   if (parent→left == currentChild)
      return RBTreeSetChild(parent, "left", newChild)
   else if (parent→right == currentChild)
      return RBTreeSetChild(parent, "right", newChild)
   return false
}
```

PROBLEM 2 8 POINTS

For each $N \in \{10^2, 10^3, 10^4, 10^5\}$, perform the following 10 times: generate N random doubles in the interval [0, 1] and add them one at a time to both a red-black tree. Record the *total wall clock time* to perform all the insertions and the *final height* of the red-black tree after all elements have been inserted. Next, add them one at a time to an AVL tree, recording the total wall clock time and final height of the AVL tree.

Repeat the above procedure for two different types of vectors:

- *Sorted* vectors of length N. You can create these by generating a *random vector* and then sorting it using an efficient sorting algorithm. *Don't include the sorting time in your wall clock time!*
- *Partially sorted* vectors of length N. Create a random vector of length N and sort the middle 50% of the vector so that the first 25% are random, followed by 50% sorted, and ending with 25% random.

For each N and each type of vector, compare the *average wall clock time and final height* of the red-black tree and the AVL tree over all 10 generated vectors. How do the heights compare to what would be expected by average-case big O complexities? Based on the wall clock times, which type of self-balancing tree would you recommend for each of the 3 types of vectors?